# 1. Problem Definitions

Computer Architecture / Digital Design

**Note: These problem definitions are AFTER I iterated on them and refined them during development to achieve the target pass rate for each one. I noticed that to avoid trip ups on syntax errors, which don't tell us much about how the model is actually thinking, I had to be explicit in providing things like specific ports or naming conventions. I included the full requirements/deliverables/specifications to provide more transparency on what was actually provided to the LLM.**

**P1: 4-bit Carry-Lookahead Adder**

**Problem:**

Design a 4-bit Carry-Lookahead Adder (CLA) in Verilog using structural modeling.

**Requirements:**

- Implement using full adders and carry lookahead logic
- Use generate statements for structural composition
- Input: two 4-bit operands (A[3:0], B[3:0])
- Input: cin (carry in)
- Output: sum[3:0] (4-bit sum)
- Output: cout (carry out)
- The adder should produce correct results for all 256 input combinations

**Deliverables:**

- Structural Verilog module named 'cla_4bit' with exact ports: input [3:0] A, input [3:0] B, input cin, output [3:0] sum, output cout
- Do not use behavioral modeling (no always @ blocks for combinational logic)
- Use proper hierarchical design with generate blocks
- Must use Verilog-2005 (NOT SystemVerilog)

**Specifications:**

- port_list: Input: A[3:0], B[3:0], cin; Output: sum[3:0], cout
- structural: Uses generate statements, no behavioral always @ blocks
- all_combinations: Correct for all 256 input combinations
- hierarchical: Composed of full adder modules

**Testing-driven Success:**

- Module compiles without errors
- Correct sum for random test vectors
- Correct carry out propagation

**Difficulty:**

This is a textbook problem with small scope, well-defined structure, and a simple algorithm. The LLM should be able to produce a correct solution most of the time.

**P2: VGA Controller**

**Problem:**

Design a VGA Controller for 640x480 resolution at 60Hz refresh in Verilog.

**Requirements:**

- Generate correct horizontal and vertical sync signals
- Input: clk (pixel clock - 25MHz or scaled equivalent)
- Input: rst_n (active-low asynchronous reset)
- Output: hsync (horizontal sync, active-low)
- Output: vsync (vertical sync, active-low)
- Output: video_on (active-high when in active video region)
- Output: pixel_x[9:0] (current x position, 0-639)
- Output: pixel_y[9:0] (current y position, 0-479)

Timing for 640x480@60Hz:

- Horizontal: 800 pixel clocks per line (including blanking)
  - Active: 640 pixels
  - Front porch: 16
  - Sync pulse: 96
  - Back porch: 48
- Vertical: 525 lines per frame
  - Active: 480 lines
  - Front porch: 10
  - Sync pulse: 2
  - Back porch: 33

**Deliverables:**

- Module named 'vga_controller' with exact ports above
- Correct sync pulse widths and positions
- Proper active video region timing
- Use scaled clock (1 cycle = 1 us) for simulation efficiency
- Must use Verilog-2005 (NOT SystemVerilog)

**Specifications:**

- port_list: Input: clk, rst_n; Output: hsync, vsync, video_on, pixel_x[9:0], pixel_y[9:0]
- h_timing: 800 cycles per line (active=640, sync=96, porch=64)
- v_timing: 525 lines per frame (active=480, sync=2, porch=43)
- sync_polarity: Both hsync and vsync active-low
- scaled_clock: Uses 1MHz clock (1us per cycle) for simulation

**Success Criteria:**

- Module compiles without errors
- HSYNC period = 800 cycles
- VSYNC period = 525 lines

- Active region positioned correctly (after sync pulses)
- video_on high only in active 640x480 region

**Difficulty:**

This is another common problem, but more advanced and involved than the CLA. This requires the LLM to think about timing and more complicated input parameters. However, it's still a textbook pattern and we give the model a lot of instruction to make sure it delivers what we want.

**P3: I2C Master Controller**

**Problem:**

Design a complete I2C Master Controller in Verilog supporting all standard I2C operations.

**Requirements:**

- Master initiates all transactions, slave device at 7-bit address 0x50 (EEPROM simulation)
- Input: clk (100MHz system clock)
- Input: rst_n (active-low asynchronous reset)
- Input: start (pulse high to begin transaction)
- Input: [7:0] data_tx (byte to send to slave)
- Input: rw (0=write, 1=read)
- Output: [7:0] data_rx (byte received from slave)
- Output: sda (I2C data line)
- Output: scl (I2C clock line)
- Output: busy (high during transaction)
- Output: error (high on NACK or failure)
- Generate START condition at beginning
- Send 7-bit slave address + R/W bit
- Wait for ACK from slave (check SDA during ACK bit)
- If write: send data byte, wait for ACK
- If read: receive byte, send ACK (except last byte sends NACK)
- Generate STOP condition at end
- Use clock stretching (wait for SCL high before proceeding)
- Open-drain outputs (SDA and SCL)

**Deliverables:**

- Module named 'i2c_master' with exact ports above
- Proper I2C START/STOP condition generation
- ACK/NACK detection and handling
- 100kHz SCL generation from 100MHz clock (divide by 1000)
- Error flag on NACK or bus fault
- Must use Verilog-2005 (NOT SystemVerilog)

**Specifications:**

- port_list: Input: clk, rst_n, start, data_tx[7:0], rw; Output: data_rx[7:0], sda, scl, busy, error
- protocol: Full I2C master (START, address, R/W, ACK, data, STOP)
- slave_addr: 0x50 (7-bit)
- clock_stretch: Support clock stretching from slave
- ack_check: Detect NACK and set error flag
- sda_scl_od: Open-drain outputs for SDA and SCL

**Success Criteria:**
- Module compiles without errors
- Correct START condition generation
- Correct 7-bit address + R/W transmission
- ACK/NACK detection working
- STOP condition at end of transaction
- Error flag set on NACK

**Difficulty:**
Designing an I2C controller is a step above the previous problem because it requires managing more complicated signals and timing logic. The state machine logic is much more complex, which should hopefully make the LLM struggle more compared to P1 and P2.

**P4: AXI Stream FIFO**
**Problem:**
Design an AXI Stream FIFO with packet boundary tracking in Verilog.
**Requirements**:
- Implement a streaming FIFO with AXI-Stream protocol interfaces
- Input: clk (AXI clock)
- Input: rst_n (active-low asynchronous reset)
- Input: s_valid (sender valid)
- Input: s_ready (sender ready - from downstream)
- Input: s_last (packet boundary - asserted on last beat of packet)
- Input: [31:0] s_data (input data)
- Input: s_strb (byte strobe, 4 bits)
- Output: m_valid (output valid)
- Output: m_ready (output ready - from upstream)
- Output: m_last (output packet boundary)
- Output: [31:0] m_data (output data)
- Output: m_strb (byte strobe, 4 bits)
- Output: [3:0] packet_count (current packet length)
- Handshaking: valid AND ready must be high for data transfer
- When s_valid & s_ready: capture s_data, s_last, s_strb

- When m_valid & m_ready: release data to downstream
- packet_count tracks size of current packet
- Store up to 16 data beats
- Full when 16 entries, empty when 0 entries
- Must preserve packet boundaries

**Deliverables:**

- Module named 'axi_stream_fifo' with exact ports above
- Proper AXI-Stream handshaking on both interfaces
- Packet boundary preservation (no splitting or merging packets)
- packet_count output accurate within packet
- Overflow protection (backpressure when full)
- Underflow prevention (m_valid low when empty)
- Must use Verilog-2005 (NOT SystemVerilog)

**Specifications:**

- port_list: Input: clk, rst_n, s_valid, s_ready, s_last, s_data[31:0], s_strb[3:0]; Output: m_valid, m_ready, m_last, m_data[31:0], m_strb[3:0], packet_count[3:0]
- protocol: AXI-Stream with packet boundaries
- depth: 16 beats
- handshake: valid/ready on both interfaces
- packet_tracking: Track and output packet size

**Success Criteria:**

- Module compiles without errors
- AXI-Stream handshaking works both directions
- Packet boundaries preserved (no truncation)
- packet_count accurate
- No data loss on overflow
- No invalid data on underflow

**Difficulty:**

This problem requires the model to try and think across both directions of handshaking, as well as multiple clock domains.

**P5: AXI4 Full Read/Write Master**

**Problem:**

Design a complete AXI4 Full Master interface in Verilog for controlling external memory.

**Requirements:**

- Implement an AXI4 Full Master that can perform read and write transactions to memory-mapped devices
- Input: clk (AXI clock), rst_n (active-low asynchronous reset), start (pulse to initiate transaction)

- Input: [31:0] addr (transaction address, must be 32-bit aligned)
- Input: [31:0] wdata (write data)
- Input: [1:0] size (0=byte, 1=halfword, 2=word)
- Input: write_en (1=write, 0=read)
- Output: [31:0] rdata (read data), valid (high when rdata is valid), ready (high when ready for new transaction)
- AXI4 address and data channels with full handshaking
- Handle all 5 AXI channels (AW, W, B, AR, R) independently
- Generate unique IDs for each transaction
- Wrap bursts NOT required (use INCR only)
- Maximum burst length: 16 beats
- Handle backpressure on all channels
- Wait for write response before completing write transaction
- Wait for read data before completing read transaction
- Proper handshaking on all channels (valid/ready protocol)
- Support byte, halfword, and word accesses via size and wstrb

**Deliverables:**

- Module named 'axi_master' with exact port list (see full spec in common.py)
- Independent state machines for AW/W/B and AR/R channels
- Proper AXI handshaking on all 5 channels
- ID tracking for outstanding transactions
- Error handling for DECERR (slave response)
- Must use Verilog-2005 (NOT SystemVerilog)

**Specifications:**

- port_list: Input: clk, rst_n, start, addr[31:0], wdata[31:0], size[1:0], write_en, arready, awready, wready, bid[3:0], bresp[1:0], bvalid, rid[3:0], rdata_in[31:0], rresp[1:0], rvalid; Output: rdata[31:0], valid, ready, arid[3:0], araddr[31:0], arvalid, awid[3:0], awaddr[31:0], awvalid, wdata_out[31:0], wstrb[3:0], wvalid, bready, rready
- protocol: AXI4 Full with 5 independent channels
- burst_type: INCR only, max 16 beats
- handshake: valid/ready on all channels
- error_handling: Handle DECERR responses

**Success Criteria:**

- Module compiles without errors
- AW/W channel handshaking works
- B channel response handling works
- AR/R channel handshaking works
- Write completes only after BVALID
- Read data captured after RVALID
- No deadlocks or livelocks

**Difficulty:**
This is a complex protocol to implement, and each channel needs its own state machine. The LLM might generate functional code, but it probably won't comply with AXI standards.

## 2. Verifiers

The verifiers and expected pass/fail cases for each question are in the verifier.py python script.

## 3. Baseline

| Problem | T1 | T2 | T3 | T4 | T5 | Pass Rate |
|---|---|---|---|---|---|---|
| P1 | **P** | **P** | **P** | **P** | **P** | 5 |
| P2 | **P** | **P** | **P** | **P** | **P** | 5 |
| P3 | **P** | F | **P** | **P** | F | 3 |
| P4 | F | F | F | F | F | 0 |
| P5 | F | F | F | F | F | 0 |

## 4. Tool Pipeline

Selection Option: A

Tool Description: 'simulation'

I made a simulation tool that allows the model to simulate its generated verilog module with a custom testbench. It can then see the results of that testbench simulation in future turns to improve on its output. This uses the same icarus verilog setup that we use for verification, and will give the model more insight into the runtime behavior of its code.

Worked Example:

```shell
(venv) arnav@Wii:~/ece498hw3$ python3 tool_pipeline.py
============================================================
Tool-Augmented Pipeline for P3 (I2C Master Controller)
```

```
============================================================

============================================================
Solving P3: I2C Master Controller
============================================================

[INITIAL] Step 1: Getting initial solution via instructor...
----------------------------------------
[INITIAL] Got initial code (12773 chars)
[INITIAL] Code preview:
module i2c_master(
    input clk,
    input rst_n,
    input start,
    input [7:0] data_tx,
    input rw,
    output reg [7:0] data_rx,
    output sda,
    output scl,
    output reg busy,
    output reg error
);

// I2C master implementing START, STOP, address, R/W, ACK/NACK, read/write byte
// Sl...

[TOOL LOOP] Step 2: Tool-calling loop...
----------------------------------------
[TOOL LOOP 1] Sending request to LLM...
[TOOL LOOP 1] LLM called tool: run_verilog_simulation
[TOOL] Module code: // i2c_master.v
// Verilog-2005
module i2c_master(
    input clk,
    input rst_n,
    input start,
...
[TOOL] Testbench: // Testbench for i2c_master
`timescale 1ns/1ps
module tb;
    reg clk;
    reg rst_n;
    reg start;...
[TOOL] Executing simulation...
```

```
[TOOL] Simulation error: Command '['vvp', '/tmp/tmp_2jsetve/sim.vvp']' timed
out after 10 seconds
[TOOL] Result: passed=False
[TOOL] Errors: ["Command '['vvp', '/tmp/tmp_2jsetve/sim.vvp']' timed out after
10 seconds"]
[TOOL LOOP 1] Simulation failed, continuing loop
[TOOL LOOP 2] Sending request to LLM...
[TOOL LOOP 2] LLM called tool: run_verilog_simulation
[TOOL] Module code: // i2c_master.v - Verilog-2005
module i2c_master(
    input clk,
    input rst_n,
    input start,
 ...
[TOOL] Testbench: `timescale 1ns/1ps
module tb;
    reg clk;
    reg rst_n;
    reg start;
    reg [7:0] data_tx;
    ...
[TOOL] Executing simulation...
[TOOL] Result: passed=True
[TOOL LOOP 2] Simulation passed, using this code

[FINAL] Step 3: Parsing final content via instructor...
----------------------------------------
[FINAL] Got structured solution: ['i2c_master_module']
[FINAL] Code preview:
// i2c_master.v - Verilog-2005
module i2c_master(
    input clk,
    input rst_n,
    input start,
    input [7:0] data_tx,
    input rw,
    output reg [7:0] data_rx,
    output sda,
    output scl,
...

============================================================
[VERIFY] Running verifier...
============================================================
```

```
[VERIFY] Result: pass=True
[VERIFY] Verification passed

============================================================
FINAL OUTPUT
============================================================
{
  "i2c_master_module": "// i2c_master.v - Verilog-2005\nmodule i2c_master(\n
input clk,\n    input rst_n,\n    input start,\n    input [7:0] data_tx,\n
input rw,\n    output reg [7:0] data_rx,\n    output sda,\n    output scl,\n
output reg busy,\n    output reg error\n);\n\n// Generate 100kHz SCL from
100MHz clock -> divide by 1000 (period). Half period = 500 cycles\nlocalparam
integer HALF_PERIOD = 16'd500;\n\n// I2C 7-bit slave address 0x50 =>
7'b1010000\nlocalparam [6:0] SLAVE_ADDR = 7'b1010000;\n\n// FSM
states\nlocalparam IDLE = 4'd0;\nlocalparam START = 4'd1;\nlocalparam SEND_BYTE
= 4'd2;\nlocalparam ACK_WAIT = 4'd3;\nlocalparam WRITE_BYTE = 4'd4;\nlocalparam
READ_BYTE = 4'd5;\nlocalparam SEND_ACK_BIT = 4'd6;\nlocalparam STOP =
4'd7;\nlocalparam DONE = 4'd8;\nlocalparam ERROR_ST = 4'd9;\n\nreg [3:0]
state;\n\n// Master-driven open-drain signals: 1 = drive low, 0 = release
(drive high)\nreg sda_drive_low; // when 1 -> drive 0 on SDA, when 0 -> release
(SDA = 1)\nreg scl_drive_low; // when 1 -> drive 0 on SCL, when 0 -> release
(SCL = 1)\n\n// Output assignments (treat release as high via pull-up)\nassign
sda = (sda_drive_low ? 1'b0 : 1'b1);\nassign scl = (scl_drive_low ? 1'b0 :
1'b1);\n\n// Read back lines (since no external pull-downs in module, these
equal outputs)\nwire sda_in = sda;\nwire scl_in = scl;\n\n// Clock divider for
SCL toggling intention\nreg [15:0] clk_cnt;\nreg scl_toggle_req;\nreg
scl_int_release; // intended state: 1 = release/high, 0 = drive low\n\nalways
@(posedge clk or negedge rst_n) begin\n    if (!rst_n) begin\n        clk_cnt
<= 16'd0;\n        scl_toggle_req <= 1'b0;\n    end else begin\n        if
(clk_cnt + 16'd1 >= HALF_PERIOD) begin\n            clk_cnt <= 16'd0;\n
scl_toggle_req <= 1'b1;\n        end else begin\n            clk_cnt <= clk_cnt
+ 16'd1;\n            scl_toggle_req <= 1'b0;\n        end\n    end\nend\n\n//
Update intended SCL on toggle requests. Actual scl_drive_low follows
intent.\nalways @(posedge clk or negedge rst_n) begin\n    if (!rst_n) begin\n
scl_int_release <= 1'b1; // idle high\n        scl_drive_low <= 1'b0; //
release\n    end else begin\n        if (scl_toggle_req) begin\n
scl_int_release <= ~scl_int_release;\n            if (~scl_int_release) begin\n
// going to release (intend high)\n                scl_drive_low <= 1'b0;\n
end else begin\n                // going to drive low\n
scl_drive_low <= 1'b1;\n            end\n        end\n        // If slave
stretches clock (external), scl_in could be held low even if released\n
// FSM must check scl_in before proceeding, but here scl equals master's drive
since no external slave\n    end\nend\n\n// Edge detection of SCL actual
line\nreg prev_scl_in;\nalways @(posedge clk or negedge rst_n) begin\n    if
```

```
(!rst_n) prev_scl_in <= 1'b1;\n    else prev_scl_in <= scl_in;\nend\nwire
scl_rising = (scl_in == 1'b1) && (prev_scl_in == 1'b0);\nwire scl_falling =
(scl_in == 1'b0) && (prev_scl_in == 1'b1);\n\n// Transaction registers\nreg
[7:0] tx_shift;\nreg [3:0] bit_cnt;\nreg rw_latched;\nreg start_req;\nreg
start_prev;\n\nalways @(posedge clk or negedge rst_n) begin\n    if (!rst_n)
begin\n        start_prev <= 1'b0;\n        start_req <= 1'b0;\n    end else
begin\n        start_prev <= start;\n        if (start && ~start_prev && ~busy)
start_req <= 1'b1;\n        else if (state != IDLE) start_req <= 1'b0;\n
end\nend\n\nalways @(posedge clk or negedge rst_n) begin\n    if (!rst_n)
begin\n        state <= IDLE;\n        sda_drive_low <= 1'b0;\n        busy <=
1'b0;\n        error <= 1'b0;\n        tx_shift <= 8'd0;\n        bit_cnt <=
4'd0;\n        data_rx <= 8'd0;\n        rw_latched <= 1'b0;\n    end else
begin\n        case (state)\n            IDLE: begin\n                error <=
1'b0;\n                sda_drive_low <= 1'b0; // release SDA\n
busy <= 1'b0;\n                if (start_req) begin\n                    busy
<= 1'b1;\n                    rw_latched <= rw;\n                    // prepare
address+R/W\n                    tx_shift <= {SLAVE_ADDR, rw};\n
bit_cnt <= 4'd8;\n                    // ensure SDA high before START\n
sda_drive_low <= 1'b0; // release\n                    state <= START;\n
end\n            end\n            START: begin\n                // START: SDA
goes low while SCL high\n                // Ensure SCL is high (bus idle) then
pull SDA low\n                sda_drive_low <= 1'b1; // drive SDA low\n
if (scl_in == 1'b1) begin\n                    // wait for next falling edge to
start clocking data\n                    if (scl_falling) begin\n
// place MSB on SDA (drive low for 0, release for 1)\n
sda_drive_low <= ~tx_shift[7];\n                        state <= SEND_BYTE;\n
end\n                end\n            end\n            SEND_BYTE: begin\n
// On SCL falling, prepare data. On SCL rising, shift.\n                if
(scl_falling) begin\n                    if (bit_cnt > 0) sda_drive_low <=
~tx_shift[7];\n                end\n                if (scl_rising) begin\n
if (bit_cnt > 0) begin\n                        // bit was sampled by slave\n
tx_shift <= {tx_shift[6:0], 1'b0};\n                        bit_cnt <= bit_cnt
- 4'd1;\n                    end\n                end\n                // After
all bits shifted out, on next falling edge release SDA for ACK\n
if ((bit_cnt == 4'd0) && scl_falling) begin\n                    sda_drive_low
<= 1'b0; // release SDA for ACK from slave\n                    state <=
ACK_WAIT;\n                end\n            end\n            ACK_WAIT: begin\n
// Sample ACK on SCL rising\n                if (scl_rising) begin\n
if (sda_in == 1'b0) begin\n                        // ACK received\n
error <= 1'b0;\n                        if (rw_latched == 1'b0) begin\n
// write: send data_tx\n                            tx_shift <= data_tx;\n
bit_cnt <= 4'd8;\n                            state <= WRITE_BYTE;\n
end else begin\n                            // read: prepare to receive\n
bit_cnt <= 4'd8;\n                            data_rx <= 8'd0;\n
```

```
sda_drive_low <= 1'b0; // release SDA to let slave drive\n
state <= READ_BYTE;\n                        end\n                    end else
begin\n                        // NACK\n                        error <=
1'b1;\n                        state <= ERROR_ST;\n                    end\n
end\n            end\n            WRITE_BYTE: begin\n                if
(scl_falling) begin\n                    if (bit_cnt > 0) sda_drive_low <=
~tx_shift[7];\n                end\n                if (scl_rising) begin\n
if (bit_cnt > 0) begin\n                        tx_shift <= {tx_shift[6:0],
1'b0};\n                        bit_cnt <= bit_cnt - 4'd1;\n
end\n                end\n                if ((bit_cnt == 4'd0) && scl_falling)
begin\n                    sda_drive_low <= 1'b0; // release for ACK\n
state <= ACK_WAIT;\n                end\n            end\n
READ_BYTE: begin\n                // Sample bits on SCL rising\n
if (scl_rising) begin\n                    data_rx <= {data_rx[6:0], sda_in};\n
bit_cnt <= bit_cnt - 4'd1;\n                end\n                if ((bit_cnt
== 4'd0) && scl_falling) begin\n                    // After last bit, send
NACK (release SDA high) and then STOP\n                    sda_drive_low <=
1'b0; // NACK = 1 -> release\n                    state <= SEND_ACK_BIT;\n
end\n            end\n            SEND_ACK_BIT: begin\n                // Clock
ACK/NACK bit on next rising edge then STOP\n                if (scl_rising)
begin\n                    state <= STOP;\n                end\n
end\n            STOP: begin\n                // Ensure SDA low then release
while SCL high to generate STOP\n                if (scl_falling) begin\n
sda_drive_low <= 1'b1; // pull low\n                end\n                if
(scl_rising) begin\n                    sda_drive_low <= 1'b0; // release while
SCL high -> STOP\n                    state <= DONE;\n                end\n
end\n            DONE: begin\n                busy <= 1'b0;\n
sda_drive_low <= 1'b0;\n                state <= IDLE;\n            end\n
ERROR_ST: begin\n                // Try to generate STOP then go to IDLE\n
if (scl_falling) begin\n                    sda_drive_low <= 1'b1;\n
end\n                if (scl_rising) begin\n                    sda_drive_low
<= 1'b0;\n                    busy <= 1'b0;\n                    state <=
IDLE;\n                end\n            end\n            default: state <=
IDLE;\n        endcase\n    end\nend\n\nendmodule\n"
}
```

## 5. Tool Evaluation

| Problem | T1 | T2 | T3 | T4 | T5 | Pass Rate |
|---|---|---|---|---|---|---|

| | | | | | | |
|---|---|---|---|---|---|---|
| P1 | **P** | **P** | **P** | **P** | **P** | 5 |
| P2 | **P** | **P** | **P** | **P** | **P** | 5 |
| P3 | **P** | **P** | **P** | F | F | 3 |
| P4 | F | F | **P** | **P** | F | 2 |
| P5 | F | F | F | **P** | F | 1 |

Adding the tool did allow the model to improve and pass the verifier, even for problem 5. I used a temperature setting of 1, so I think with only one chance to produce a correct result the LLM still struggled for the harder problems, although better than the baseline.

## 6. Self Refinement

| Trial | Turn 1 | Turn 2 | Turn 3 | Final |
|---|---|---|---|---|
| 1 | Fail | Pass | — | Pass |
| 2 | Fail | Fail | Pass | Pass |
| 3 | Fail | Fail | Pass | Pass |
| 4 | Fail | Pass | — | Pass |
| 5 | Fail | Fail | Pass | Pass |

Yes. Refinement alone was pretty ineffective, since the LLM had no way to get answers for its “unanswered questions”. For example, all it would know is which testcase it’s failing. It was hard for the model to understand what specific pieces of code were wrong, and why they were causing the test case to fail. Giving it the simulation tool allowed it to test out different things to see exactly *why* it was wrong, and how it can improve for the future. I noticed that we don’t need multiple turns of refinement, since in trial 1 we could achieve a passing solution after just a single refinement. We were able to get an eventual pass across 5 independent trials, so both in conjunction were very effective.